Pose Forecasting

Abstract

Pose forecasting aims to predict future human body poses from a sequence of past observations. In this paper, I investigate multiple methods for autoregressive human pose forecasting in a unified latent space, including zero-velocity, constant-velocity, MLP, RNN, and Transformer-based models. To obtain a compact and geometrically meaningful representation of human motion, I first encode input pose sequences using a pretrained VPoser model and perform forecasting in the resulting latent space. I conduct experiments under five observation-prediction settings covering both short-term and long-horizon forecasting scenarios: (a) given 0.5 seconds of observation, the model predicts 1 second; (b) given 1 second of observation, the model predicts 1, 2, and 3 seconds; and (c) given 2 seconds of observation, the model predicts 3 seconds. All methods are evaluated using standard motion forecasting metrics, including average displacement error (ADE), final displacement error (FDE), and mean per-joint position error (MPJPE), across multiple forecasting horizons. The source code and pre-trained models will be publicly available at https://github.com/mlnjeongpark/Pose-Forecasting-Boilerplate.

1. Method

1.1. Overview

As shown in Fig. 1, our framework consists of VPoser and a trainable autoregressive model. Let $X = [X_{-O+1}, X_{-O+2}, \ldots, X_0, X_1, \ldots, X_P] \in \mathbb{R}^{(O+P)\times 63}$ be a clean complete normalized sequence of human body poses in O frames of observation and P frames of future. The observed sequence $\{X_{-O+1}, X_{-O+2}, \cdots, X_0\} \in \mathbb{R}^{O\times 63}$ is processed by the VPoser encoder to obtain the VPoser-embedded feature $\tilde{X} \in \mathbb{R}^{O\times 32}$. The feature is then passed through a trainable autoregressive model to predict

$$\hat{z} = \{\hat{z}_1, \hat{z}_2, \cdots, \hat{z}_P\} \in \mathbb{R}^{P\times 32}.$$

I supervise $\hat{z}$ with an MSE loss against the VPoser-encoded embedding of the target pose. The predicted feature $\hat{z}$ is then decoded by the VPoser decoder to obtain z, and the model is further optimized using L_{MPJPE} between the decoded poses and the target poses $X_1, \ldots, X_P$.

1.2. Trainable Autoregressive Model Choice

To model future human motion, I explore a diverse set of autoregressive models along with simple non-learned baselines in a unified latent space. The objective is to systematically compare heuristic methods with trainable neural architectures under a consistent formulation.

1.2.1. Non-Learned Baselines.

I begin with two simple yet effective baselines defined in the latent space.

Zero-Velocity. The zero-velocity baseline assumes that the subject remains static after the observation window. The future latent states are obtained by repeating the last observed latent vector:

$$\hat{\mathbf{z}}_{T+k} = \mathbf{z}_T, \quad \forall k \in \{1, \ldots, K\}. \tag{1}$$

Constant-Velocity. The constant-velocity baseline extrapolates motion using the last observed velocity in latent space. Let $\mathbf{v} = \mathbf{z}_T - \mathbf{z}_{T-1}$ denote the estimated velocity. Future states are computed as:

$$\hat{\mathbf{z}}_{T+k} = \mathbf{z}_T + k \cdot \mathbf{v}. \tag{2}$$

These baselines provide strong references for short-term prediction and serve as lower bounds for learned models.

1.2.2. Trainable Autoregressive Models.

I further investigate several trainable autoregressive models that predict future motion iteratively in latent space. All models follow a unified formulation: given a context window of length T, they predict the next latent state and update the context in a sliding-window manner.

MLP-based Autoregressive Model. The MLP model predicts the next latent vector by flattening the entire context window:

$$\hat{\mathbf{z}}_{T+1} = f_{\mathrm{MLP}}(\mathbf{z}_{1:T}). \tag{3}$$

The predicted latent is appended to the sequence, and the oldest element is removed to maintain a fixed-length context. Although simple, this model captures temporal dependencies implicitly through the aggregated representation of the input sequence.

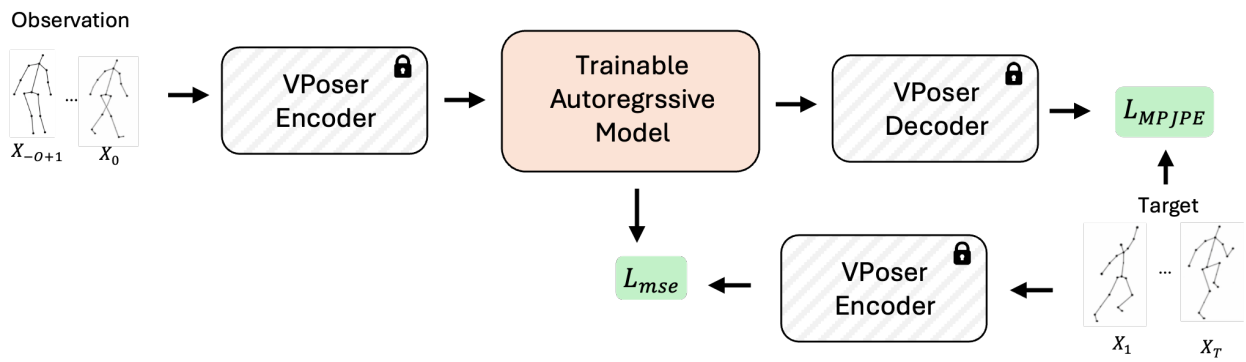

Figure 1. Illustration of the training pipeline.

Recurrent Neural Network (RNN). I employ a recurrent neural network to explicitly model temporal dependencies. The observed latent sequence is first processed to produce a hidden state:

$$\mathbf{h}_T = \mathrm{RNN}(\mathbf{z}_{1:T}), \tag{4}$$

and future latents are generated autoregressively:

$$\hat{\mathbf{z}}_{T+k}, \mathbf{h}_{T+k} = \mathrm{RNN}(\hat{\mathbf{z}}_{T+k-1}, \mathbf{h}_{T+k-1}). \tag{5}$$

This formulation allows the model to maintain a compact temporal representation and capture sequential dynamics more effectively than the MLP.

Transformer-based Model. I also adopt a Transformer-based autoregressive model to capture long-range dependencies. The observed latent sequence is embedded and processed using causal self-attention. During training, I apply teacher forcing by shifting the target sequence, while at inference time, predictions are generated sequentially:

$$\hat{\mathbf{z}}_{T+1:T+K} = \mathrm{Transformer}(\mathbf{z}_{1:T}). \tag{6}$$

The self-attention mechanism enables flexible modeling of temporal relationships across the entire observation window.

1.3. Optimization

To train the forecasting model, I optimize both the prediction accuracy in the latent space and the geometric consistency in the decoded pose space. Specifically, I use a combination of an MSE loss in the VPoser latent space and an MPJPE-based loss in the decoded pose space.

Huber Loss. Let $\hat{\mathbf{z}} = \{\hat{\mathbf{z}}_1, \hat{\mathbf{z}}_2, \ldots, \hat{\mathbf{z}}_P\}$ denote the predicted future latent sequence and $\mathbf{z} = \{\mathbf{z}_1, \mathbf{z}_2, \ldots, \mathbf{z}_P\}$ denote the corresponding ground-truth latent sequence obtained from the pretrained VPoser encoder. I first supervise the predicted latent vectors using Huber loss:

$$\mathcal{L}_{\text{Huber}} = \mathcal{L}(\hat{\mathbf{z}}, \mathbf{z}) \tag{7}$$

This loss encourages the autoregressive model to accurately predict future motion trajectories in the compact latent space, where interpolation and smooth temporal transitions are more geometrically meaningful.

MPJPE Loss. Although latent-space supervision is effective for training, the final objective of pose forecasting is to generate accurate human poses in 3D space. Therefore, I additionally decode the predicted latent sequence using the pretrained VPoser decoder and apply a mean per-joint position error (MPJPE) loss on the resulting poses. Let $\hat{\mathbf{p}}_{t,j} \in \mathbb{R}^3$ and $\mathbf{p}_{t,j} \in \mathbb{R}^3$ denote the predicted and ground-truth 3D locations of joint j at future time step t, respectively. The MPJPE loss is defined as

$$\mathcal{L}_{\text{MPJPE}} = \frac{1}{PJ} \sum_{t=1}^{P} \sum_{j=1}^{J} \|\hat{\mathbf{p}}_{t,j} - \mathbf{p}_{t,j}\|_2 . \tag{8}$$

This term directly penalizes geometric pose errors in 3D joint space and improves the physical plausibility of the decoded predictions.

Overall Objective. The final training objective is a weighted combination of the two losses:

$$\mathcal{L} = \lambda_{mse}\mathcal{L}_{MSE} + \lambda_{mpjpe}\mathcal{L}_{MPJPE}, \tag{9}$$

where λ_{mse} and λ_{mpjpe} control the relative importance of latent reconstruction accuracy and geometric pose accuracy, respectively.

2. Experiment

2.1. Implementation Details

The models are implemented with PyTorch [4] and trained on an A5000 GPU. I subsampled the frame rate down to 30 frames per second. The model is trained for 500 epochs with a batch size of 64, using the AdamW optimizer [2] and a cosine learning rate scheduler [1]. The initial learning rate is set to 1×10^{-4} and is gradually decayed to 1×10^{-6}.

Table 1. Observation and prediction horizons used for evaluation.

Setting	Observation (s)	Prediction (s)
Setting-A	0.5	1
Setting-B	1	1
Setting-C	1	2
Setting-D	1	3
Setting-E	2	3

The initial learning rate is set to 1×10^{-4} and decayed to 1×10^{-6} during training. All models use 8 layers and an embedding dimension of 512. For the Transformer-based model, I additionally use 8 attention heads. The hyperparameters λ_{mse} and λ_{mpjpe} are set to 1 and 0.1, respectively. I conducted experiments on the given dataset, which consists of pose sequences from the AMASS dataset [3]. I divided the sequences into non-overlapping training and testing sets. Specifically, I used 80% of the data for training and the remaining 20% for testing. The training samples were generated using a sliding window approach with a stride of 1. To comprehensively evaluate forecasting performance from short-term to long-horizon settings, I conducted experiments under five observation-prediction settings, as summarized in Table 1.

2.2. Quantitative Results

I report performance comparisons across five settings with zero-velocity, constant-velocity, MLP, RNN, and Transformer-based methods, as shown in Tables 2, 3, 4, 5, and 6.

Interestingly, the zero-velocity baseline performed competitively in Setting-A, where only 0.5 seconds of motion were observed and the next 1 second was predicted. This behavior is plausible for two reasons. First, short-term human motion often remains close to the last observed pose, making a conservative prediction surprisingly effective under MPJPE. Second, the shorter observation window provides limited evidence for estimating motion dynamics such as direction, phase, and acceleration, which can make trainable autoregressive models more prone to unstable extrapolation. As a result, simply repeating the last observed latent state can serve as a strong baseline in short-horizon settings, even though it becomes less effective as the prediction horizon increases.

The Transformer-based autoregressive model demonstrates strong performance, particularly in long-horizon prediction settings. Compared to simpler baselines and other trainable models, the Transformer maintains lower error as the prediction horizon increases. This suggests that its ability to model long-range temporal dependencies is especially beneficial when forecasting motion over extended time intervals.

In contrast, the MLP-based autoregressive model does not perform as well as expected across most settings. One possible explanation is that the model architecture, particularly the depth of the network, may not be well suited for this task. In our implementation, the MLP consists of multiple stacked layers operating on a flattened representation of the input sequence. While increasing depth can, in principle, improve representational capacity, it may also make optimization more difficult and lead to overfitting, especially given the relatively limited temporal structure encoded in the input. Moreover, unlike RNNs or Transformers, the MLP does not explicitly model temporal dependencies. Instead, it relies on a fixed-length flattened context, which may limit its ability to capture temporal dynamics such as motion continuity and periodicity. As the network becomes deeper, this limitation can be exacerbated, as the model may focus more on fitting local patterns in the training data rather than learning meaningful temporal transitions.

2.3. Qualitative Result

Figure 2 presents a qualitative comparison of the predicted human poses generated by different models, including MLP, RNN, and Transformer, against the ground truth (GT). While all models exhibit some deviation from the ground truth, the Transformer-based model produces predictions that are visually closer to the GT.

In particular, the MLP-based model shows noticeable discrepancies in joint configurations, likely due to its limited ability to model temporal dependencies. The RNN improves upon this by generating more coherent motion; however, some inaccuracies in pose alignment and articulation remain. In contrast, the Transformer model produces more accurate and stable predictions, especially in the overall body posture and limb positioning.

These results are consistent with the quantitative evaluation, suggesting that the Transformer's ability to capture long-range temporal dependencies leads to more reliable motion prediction. Overall, the qualitative results further support the effectiveness of the Transformer architecture in modeling complex human motion dynamics.

3. Conclusion

In this paper, I investigated a range of autoregressive models for human pose forecasting in a unified latent space. By leveraging a pretrained VPoser encoder and decoder, all models operate on a compact and smooth representation of human motion, enabling a fair comparison between simple baselines and trainable architectures.

Through extensive experiments under multiple observation and prediction settings, I demonstrated that model choice plays a critical role in forecasting performance. In short-horizon settings, simple baselines such as zero-

Model	MPJPE ↓						ADE ↓	FDE ↓
	80ms	320ms	560ms	720ms	880ms	1000ms		
Zero	0.090	0.207	0.253	0.268	0.276	0.280	0.217	0.279
Constant	0.087	0.366	0.635	0.810	0.981	1.083	0.564	1.082
MLP	0.153	0.274	0.306	0.319	0.347	0.335	0.287	0.335
RNN	0.133	0.226	0.284	0.311	0.332	0.344	0.255	0.344
Transformer	0.079	0.193	0.251	0.269	0.275	0.279	0.213	0.262

Table 2. Comparison of models under **Setting A**. 'Zero' and 'Constant' denote the zero-velocity and constant-velocity models, respectively.

Model	MPJPE ↓						ADE ↓	FDE ↓
	80ms	320ms	560ms	720ms	880ms	1000ms		
Zero	0.090	0.206	0.252	0.267	0.276	0.279	0.216	0.279
Constant	0.088	0.366	0.635	0.810	0.982	1.084	0.564	1.083
MLP	0.161	0.292	0.349	0.375	0.389	0.397	0.311	0.397
RNN	0.133	0.266	0.284	0.310	0.332	0.344	0.257	0.344
Transformer	0.142	0.237	0.273	0.285	0.293	0.297	0.247	0.297

Table 3. Comparison of models under **Setting B**. 'Zero' and 'Constant' denote the zero-velocity and constant-velocity models, respectively.

Model	MPJPE ↓						ADE ↓	FDE ↓
	80ms	320ms	560ms	720ms	880ms	1000ms		
Zero	0.089	0.204	0.250	0.265	0.275	0.280	0.254	0.308
Constant	0.087	0.362	0.628	0.802	0.973	1.108	1.076	2.078
MLP	0.162	0.265	0.299	0.313	0.325	0.333	0.282	0.333
RNN	0.134	0.222	0.280	0.302	0.322	0.336	0.312	0.390
Transformer	0.094	0.148	0.175	0.198	0.210	0.216	0.253	0.281

Table 4. Comparison of models under **Setting C**. 'Zero' and 'Constant' denote the zero-velocity and constant-velocity models, respectively.

Model	MPJPE ↓						ADE ↓	FDE ↓
	80ms	320ms	560ms	720ms	880ms	1000ms		
Zero	0.087	0.199	0.245	0.260	0.270	0.275	0.275	0.337
Constant	0.085	0.353	0.613	0.783	0.949	1.081	1.547	3.023
MLP	0.159	0.265	0.296	0.310	0.320	0.322	0.385	0.473
RNN	0.130	0.218	0.270	0.291	0.310	0.321	0.349	0.418
Transformer	0.093	0.147	0.235	0.254	0.263	0.271	0.273	0.310

Table 5. Comparison of models under **Setting D**. 'Zero' and 'Constant' denote the zero-velocity and constant-velocity models, respectively.

velocity can perform competitively, as short-term motion often remains close to the last observed pose. However, their performance degrades significantly as the prediction horizon increases.

Among the trainable models, the Transformer-based autoregressive model achieves the best performance, particularly in long-horizon scenarios. This can be attributed to its ability to capture long-range temporal dependencies through self-attention, resulting in more stable and coherent motion predictions. In contrast, the MLP-based model shows limited performance, likely due to its inability to explicitly model temporal dynamics and the challenges associated with deeper feedforward architectures. The RNN model provides a reasonable trade-off, but still falls short of

Model	MPJPE ↓						ADE ↓	FDE ↓
	80ms	320ms	560ms	720ms	880ms	1000ms		
Zero	0.086	0.196	0.240	0.255	0.265	0.271	0.271	0.333
Constant	0.083	0.347	0.603	0.770	0.933	1.063	1.523	2.979
MLP	0.173	0.258	0.284	0.296	0.306	0.312	0.323	0.381
RNN	0.128	0.215	0.270	0.299	0.320	0.335	0.345	0.418
Transformer	0.119	0.156	0.200	0.215	0.219	0.233	0.229	0.269

Table 6. Comparison of models under **Setting E**. 'Zero' and 'Constant' denote the zero-velocity and constant-velocity models, respectively.

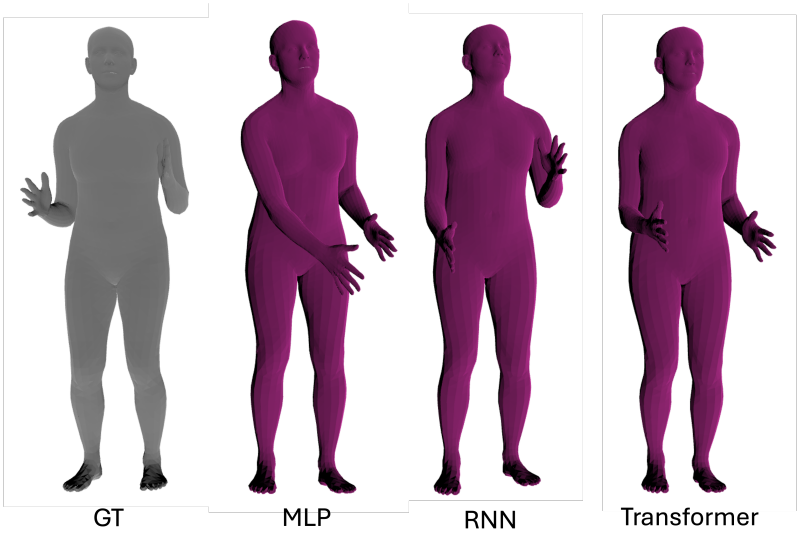

Figure 2. Comparison of the predicted human poses generated by different models, including MLP, RNN, and Transformer, against the ground truth (GT) in Setting-C.

the Transformer in capturing complex motion patterns.

Qualitative results further support these findings, showing that the Transformer produces predictions that are visually closer to the ground truth, with more accurate body posture and limb alignment.

Overall, this study highlights the importance of temporal modeling in human motion forecasting and demonstrates the effectiveness of attention-based architectures for long-horizon prediction. Future work may explore more advanced generative approaches, such as diffusion-based models, to better capture the stochastic nature of human motion.

References

[1] Ilya Loshchilov and Frank Hutter. Sgdr: Stochastic gradient descent with warm restarts. In *International Conference on Learning Representations*, 2016. 2

[2] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2017. 2

[3] Naureen Mahmood, Nima Ghorbani, Nikolaus F. Troje, Gerard Pons-Moll, and Michael J. Black. AMASS: Archive of motion capture as surface shapes. In *International Conference on Computer Vision*, pages 5442–5451, 2019. 3

[4] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. Pytorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems 32*, pages 8024–8035. Curran Associates, Inc., 2019. 2